

Задача о максимальном потоке. Теорема и алгоритм Форда-Фалкерсона

Подготовил Сивухин Никита. По вопросам пишите на почту sivukhin.work+teach@gmail.com

Поток в транспортной сети

Ориентированным графом $G = (V, E)$, где для каждой пары вершин $(u, v) \in V \times V$ задана пропускная способность $c(u, v) \geq 0$, а также выделена пара вершин s (исток) и t (сток) будем называть *транспортной сетью*.

Замечание 1. Обратите внимание, что пропускная способность задана для каждой упорядоченной пары вершин $(v, u) \in V \times V$. В таком случае если в графе нет ориентированного ребра (v, u) , то пропускная способность считается равной нулю $c(u, v) = 0$.

Потоком в сети называется функция $f : V \times V \mapsto \mathbb{R}$, удовлетворяющая следующим свойствам:

1. Антисимметричность: $f(u, v) = -f(v, u)$
2. Соответствие ограничениям: $f(u, v) \leq c(u, v)$
3. Сохранение потока: $\sum_v f(u, v) = 0$ для всех u кроме s и t

Величиной потока в сети называется сумма потока по рёбрам, выходящим из истока: $|f| = \sum_v f(s, v)$.

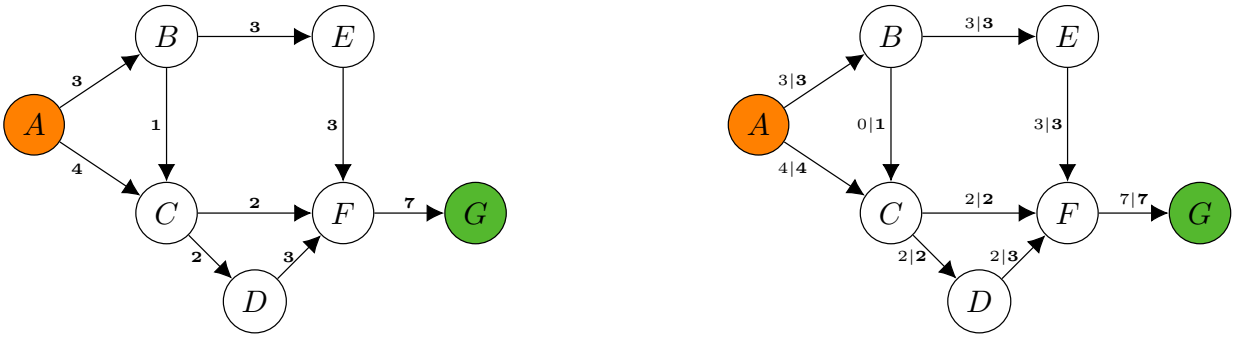


Рис. 1: Пример транспортной сети (слева) и потока в ней (справа, $|f| = 7$)

Замечание 2. Несложно показать, что для любого потока в сети верно, что $\sum_v f(s, v) = \sum_v f(v, t)$.

Также как и в случае задаче о взвешенном вершинном покрытии, существование максимального потока является неочевидным фактом, так как в общем случае множество корректных назначений функции потока для заданного графа бесконечно (т.к. поток по ребру может принимать любое действительное значение).

Лемма 1. Для любой сети $G = (V, E, c)$ существует максимальный поток $f : V \times V \mapsto \mathbb{R}$

Доказательство. Если обозначить за $c_{\max} = \max c(u, v)$ максимальное значение пропускной способности в сети, то ясно, что в любом потоке $|f(u, v)| \leq c_{\max}$.

Заметим также, что если пронумеровать пары вершин в некотором порядке, то функция f будет задаваться $m = |V|^2$ размерным вектором $f \in [-c_{\max} \dots c_{\max}]^m \subset \mathbb{R}^m$.

Тогда обозначим за F пространство всех корректных потоков $F \subseteq [-c_{\max} \dots c_{\max}]^m$ для графа $G = (V, E, c)$.

Покажем, что F — замкнутое множество. Действительно, если f' — предельная точка F , то существует последовательность $f_1, f_2, \dots \in F$ такая, что $\lim\{f_i\} = f'$, однако несложно проверить, что все три свойства потока будут сохраняться при предельном переходе, а значит $f' \in F$.

Таким образом, множество F образует компакт (замкнуто и ограничено), а значит по теореме Вейерштрасса, непрерывная линейная функция $\sum_v f(s, v)$, определенная на F , **достигает** максимума в некоторой точке F . $\square$

Разрез в транспортной сети

(s, t) -разрезом $\langle S, T \rangle$ в сети $G = (V, E, c)$ называется пара множеств S, T таких, что

1. $s \in S$ и $t \in T$
2. $T = V \setminus S$

Пропускная способность разреза обозначается как $c(S, T)$ и равна сумме пропускных способностей рёбер, соединяющих разные доли разреза: $c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$.

Поток через разрез обозначается как $f(S, T)$ и равен сумме потоков через рёбра, соединяющие разные доли разреза: $f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v)$.

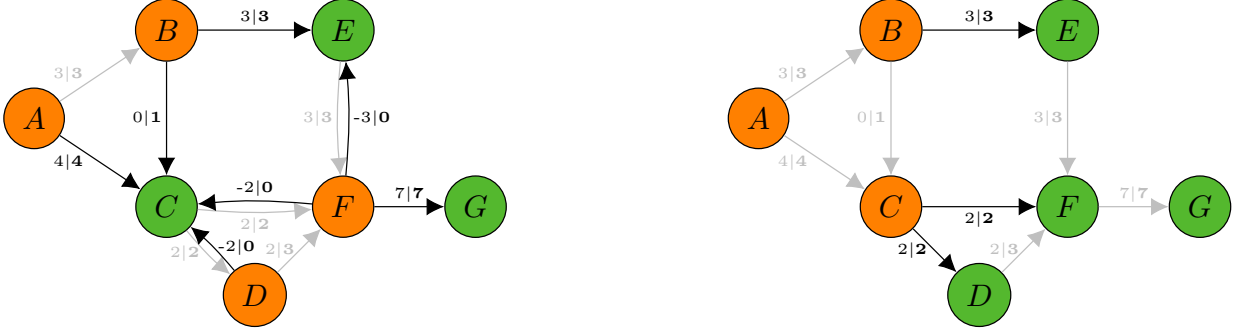


Рис. 2: Примеры разрезв в транспортной сети (доля S покрашена в оранжевый цвет, а T — в зелёный). В разрезах учитываются только рёбра из S в T . Для примера слева $c(S, T) = 15$, $f(S, T) = 7$, а справа — $c(S, T) = f(S, T) = 7$

Замечание 3. Для любого разреза $\langle S, T \rangle$ и потока f верно, что $f(S, T) \leq c(S, T)$

Лемма 2. Для любого разреза $\langle S, T \rangle$ и потока f верно, что $|f| = f(S, T)$

Доказательство. Из-за свойства сохранения потока, все $|f|$ изначальных единиц потока, пущенных из истока s , должны будут пройти по рёбрам разреза ровно один раз. Формально, доказать данную лемму можно следующим образом:

$$\begin{aligned}
 f(S, T) &= \sum_{u \in S} \sum_{v \in T} f(u, v) = \sum_{u \in S} \sum_{v \in T} f(u, v) + \underbrace{\sum_{u \in S} \sum_{v \in S} f(u, v)}_{\text{равно 0, из-за антисимметричности}} = \\
 \sum_{u \in S} \sum_{v \in V} f(u, v) &= \sum_{v \in V} f(s, v) + \underbrace{\sum_{u \in S \setminus \{s\}} \sum_{v \in V} f(u, v)}_{\text{равно 0, из-за сохранения потока}} = \sum_{v \in V} f(s, v)
 \end{aligned}$$

□

Метод Форда-Фалкерсона

Для транспортной сети $G = (V, E, c)$ и потока f можно определить *остаточную сеть* $G_f = (V, E_f, c_f)$, где пропускная способность определяется как $c_f(u, v) = c(u, v) - f(u, v)$, а рёбрами данной сети являются рёбра оригинального графа для которых остаточная пропускная способность положительна: $E_f = \{e \in E \mid c_f(e) > 0\}$. Так как f корректный поток, то $f(u, v) \leq c(u, v)$ (по второму свойству), а значит $c_f(u, v) \geq 0$.

Дополняющим путём относительно потока f называется путь $P = (p_0, p_1, p_2, \dots, p_{k-1}, p_k)$ в остаточной сети G_f из истока в сток ($p_0 = s, p_k = t$).

Замечание 4. Любому дополняющему пути всегда можно поставить в соответствие поток f' в остаточной сети G_f положительной величины $\Delta = \min\{c_f(p_i, p_{i+1})\}$:

$$f'(u, v) = \begin{cases} +\Delta & , u = p_i, v = p_{i+1} \\ -\Delta & , u = p_i, v = p_{i-1} \\ 0 & \text{иначе} \end{cases}$$

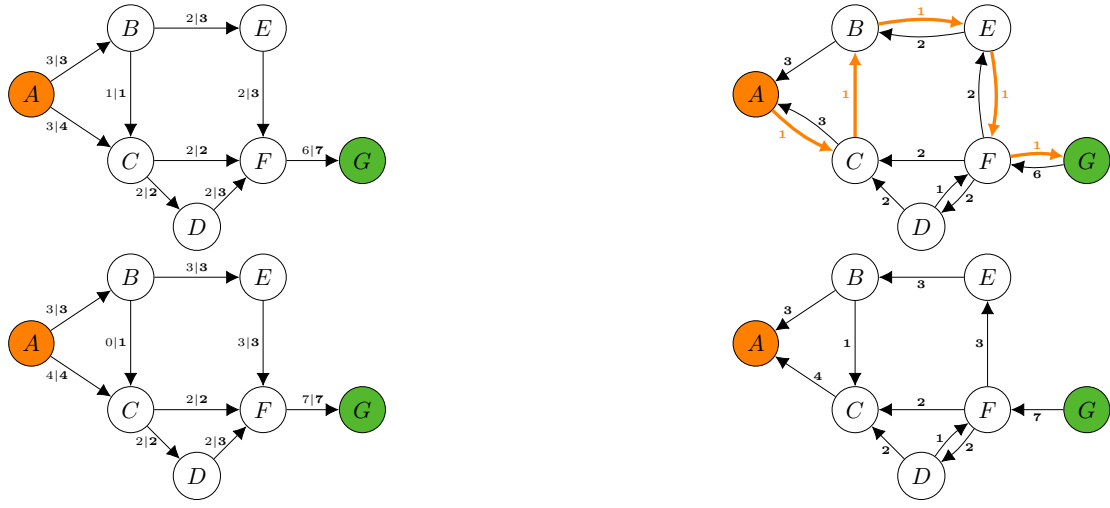


Рис. 3: На первой картинке изображён поток f , на второй — остаточная сеть G_f вместе с дополняющим путём P , на третьей — результат прибавления f' , соответствующего P , к потоку f , а на четвёртой — остаточная сеть $G_{f+f'}$

Замечание 5. Если в остаточной сети G_f есть поток f' положительной величины, то в G_f существует дополняющий путь P

Доказательство. Пусть в G_f нет дополняющего пути. Тогда, множество всех вершин S достижимых из s в G_f задают разрез $\langle S, V \setminus S \rangle = \langle S, T \rangle$. Заметим, что пропускная способность $c(S, T)$ данного разреза должна быть равна 0, потому что иначе некоторая вершина $v \in T$, должна была принадлежать доле S . Но тогда $|f'| = f'(S, T) \leq c(S, T) = 0$, что противоречит предположению, что f' имел положительную величину. $\square$

Лемма 3. (сумма потоков) Если f' поток в остаточной сети G_f , то сумма потоков $f + f'$ определяемая как $(f + f')(u, v) = f(u, v) + f'(u, v)$ является корректным потоком в изначальной сети G величины $|f| + |f'|$.

Доказательство. Покажем, что для суммы потоков выполняются все три необходимые свойства:

1. Антисимметричность: $f(u, v) = -f(v, u)$ и $f'(u, v) = -f'(v, u) \Rightarrow f(u, v) + f'(u, v) = -f(v, u) - f'(v, u)$
2. Соответствие ограничениям: так как f' поток остаточной сети G_f , то $f'(u, v) \leq c(u, v) - f(u, v)$, а значит $f(u, v) + f'(u, v) \leq c(u, v)$
3. Сохранение потока: так как f и f' корректные потоки, то для любой вершины u кроме s и t верно, что $\sum_v f(u, v) = 0$ и $\sum_v f'(u, v) = 0$, а значит $\sum_v f(u, v) + f'(u, v) = 0$

Величина потока $f + f'$ будет выражаться как $\sum_v (f(s, v) + f'(s, v)) = \sum_v f(s, v) + \sum_v f'(s, v) = |f| + |f'|$. $\square$

Лемма 4. (разность потоков) Если h, f — пара потоков в сети G , то разность потоков $h - f$ определяемая как $(h - f)(u, v) = h(u, v) - f(u, v)$ является корректным потоком в остаточной сети G_f и имеет величину величины $|h| - |f|$.

Доказательство. Покажем, что для разности потоков выполняются все три необходимые свойства:

1. Антисимметричность: $h(u, v) = -h(v, u)$ и $f(u, v) = -f(v, u) \Rightarrow h(u, v) - f(u, v) = -h(v, u) + f(v, u)$
2. Соответствие ограничениям: $h(u, v) \leq c(u, v)$, а значит $h(u, v) - f(u, v) \leq c(u, v) - f(u, v) = c_f(u, v)$
3. Сохранение потока: так как h и f корректные потоки, то для любой вершины u кроме s и t верно, что $\sum_v h(u, v) = 0$ и $\sum_v f(u, v) = 0$, а значит $\sum_v h(u, v) - f(u, v) = 0$

Величина потока $h - f$ будет выражаться как $\sum_v (h(s, v) - f(s, v)) = \sum_v h(s, v) - \sum_v f(s, v) = |h| - |f|$. $\square$

Из лемм о сумме и разности потоков можно доказать теорему Форда-Фалкерсона:

Теорема 1. Поток f максимален $\Leftrightarrow$ в остаточной сети G_f нет дополняющего пути

Доказательство.

- ($\Rightarrow$) Рассмотрим максимальный поток f и допустим, что в G_f есть дополняющий путь. Так как по любому дополняющему пути можно сконструировать поток f' положительной величины в остаточной сети, то пользуясь леммой о сумме потоков получаем, что $f + f'$ поток в G и $|f| + |f'| > |f|$, что противоречит предположению о максимальной f
- ($\Leftarrow$) Пусть в G_f нет дополняющего пути, но f — не является максимальным. Значит есть поток h такой, что $|h| > |f|$. Рассмотрим разность потоков $h - f$, которая задаёт корректный поток положительной величины в G_f . Но тогда в G_f есть дополняющий путь, что противоречит изначальному предположению.

□

Теорема 1 даёт прямой способ построения максимального потока: для этого достаточно искать дополняющий путь в остаточной сети и пропускать через него максимально возможную величину потока. Этот подход называется **методом Форда-Фалкерсона** и в худшем случае, если выбирать произвольную увеличивающуюся цепь, данный метод работает за $O(|E| \max |f|)$ для транспортных сетей с целочисленными пропускными способностями.

Лемма 5. Величина максимального потока $|f|$ равна пропускной способности минимального разреза $c(S, T)$

Доказательство. Покажем, что для максимального потока f можно конструктивно построить разрез $\langle S, T \rangle$ такой, что $c(S, T) = |f|$. Для этого рассмотрим разрез, где S — множество всех вершин, достижимых из s в остаточной сети G_f . Из теоремы 1 следует, что $t \notin S$, а значит $\langle S, V \setminus S \rangle = \langle S, T \rangle$, является корректным разрезом. Заметим, что для любой пары вершины $u \in S, v \in T$ верно, что $f(u, v) = c(u, v)$, потому что иначе в остаточной сети было бы ребро (u, v) и вершина v должна была принадлежать доле S . Но отсюда следует, что $|f| = f(S, T) = c(S, T)$.

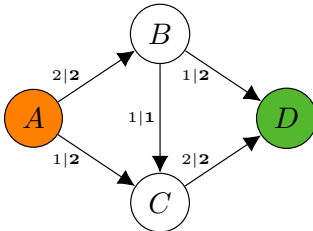
Так как для любого потока и любого разреза верно, что $f(S, T) \leq c(S, T)$, то разрез $c(S, T)$ — минимальный.

□

Реализация метода Форда-Фалкерсона

Базовой операцией в алгоритмах нахождения потока является операция сложения потоков. Так как обычно поток задается рёбрами с положительным значением потока, то возникает задача обновления противоположных рёбер с целью поддержания свойства антисимметричности.

Для эффективной реализации такой операции обычно используется следующий трюк: вместо того чтобы хранить рёбра в списках смежности соответствующих вершин, в списках смежности хранятся индексы в общем списке рёбер графа, где поддерживается инвариант, что прямое и обратное рёбра имеют индексы $2i$ и $2i + 1$ соответственно. Благодаря такому представлению рёбер, при увеличении потока вдоль ребра с индексом i на величину Δ , для поддержания свойства антисимметричности достаточно уменьшить поток вдоль ребра с индексом $i \oplus 1$ на ту же величину Δ .



index	edge	flow	capacity
0	A→B	2	2
1	B→A	-2	0
2	A→C	1	2
3	C→A	-1	0
4	B→C	1	1
5	C→B	-1	0
6	B→D	1	2
7	D→B	-1	0
8	C→D	2	2
9	D→C	-2	0

index	edge	flow	capacity
0	A→B	2	2
1	B→A	-2	0
2	A→C	2	2
3	C→A	-2	0
4	B→C	0	1
5	C→B	0	0
6	B→D	2	2
7	D→B	-2	0
8	C→D	2	2
9	D→C	-2	0

Рис. 4: Пример хранения рёбер графа и применения потока вдоль дополняющей цепи

```

1 struct Edge { int to, flow, capacity; };
2 struct Graph {
3     int n, S, T;
4     vector<Edge> edges;           // we store edges in pairs to simplify flow updates operation
5     vector<vector<int>>> graph; // graph stores indices in edges array instead of Edge structs
6     void add_direct_edge(int a, int b, int capacity) {
7         // add edges in such a way that a->b will have index 2i and b->a: 2i+1
8         graph[a].push_back(edges.size()); edges.push_back(Edge{b, 0, capacity});
9         graph[b].push_back(edges.size()); edges.push_back(Edge{a, 0, 0});
10    }
11    // try to find augmenting path and push flow_limit units of flow through it
12    bool try_augment(int v, vector<bool> &used, int &flow_limit) {
13        if (v == T) { return true; }
14        if (used[v]) { return false; }
15        used[v] = true;
16        for (int e : graph[v]) {
17            int residual = min(flow_limit, edges[e].capacity - edges[e].flow);
18            if (residual > 0 && try_augment(edges[e].to, used, residual)) {
19                flow_limit = residual;
20                edges[e].flow += residual;
21                edges[e ^ 1].flow -= residual;
22                return true;
23            }
24        }
25        return false;
26    }
27    int max_flow() {
28        auto used = vector<bool>(n);
29        int total_flow = 0;
30        while (true) {
31            std::fill(used.begin(), used.end(), 0);
32            int flow_limit = MAX_FLOW;
33            if (!try_augment(S, used, flow_limit)) { break; }
34            total_flow += flow_limit;
35        }
36        return total_flow;
37    }
38 };

```